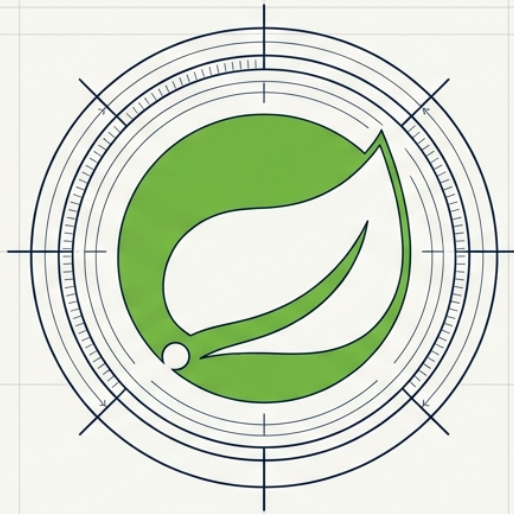
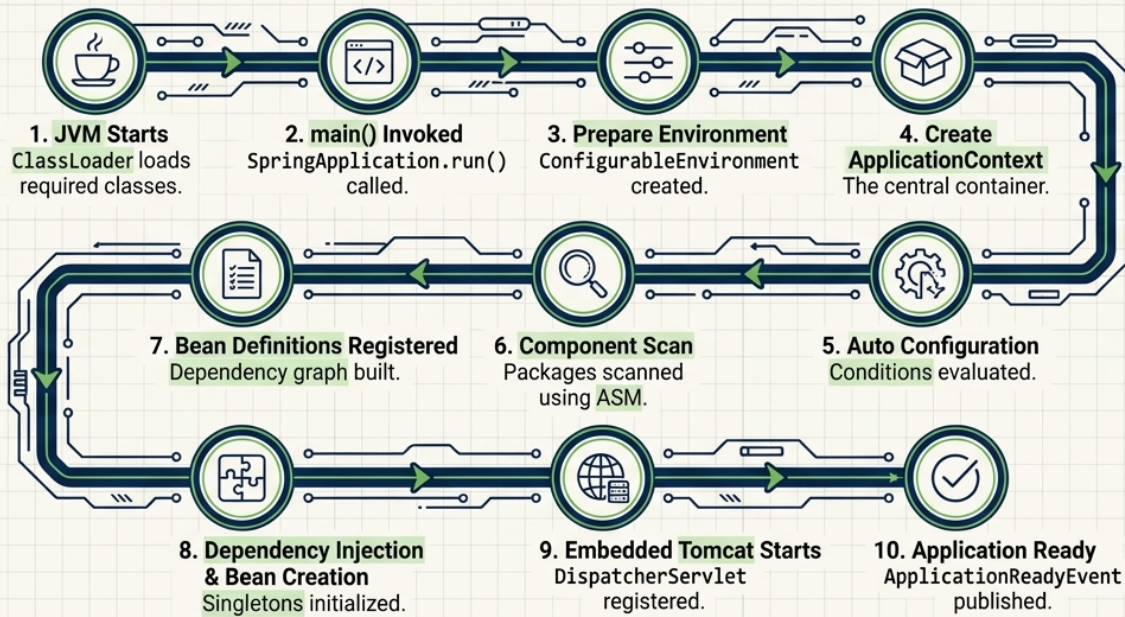




Spring Boot Under the Hood: The Startup Schematic

A technical dossier on the architecture, actors, and
lifecycle of `SpringApplication.run()`

The Macro Sequence: 10 Steps to Application Ready

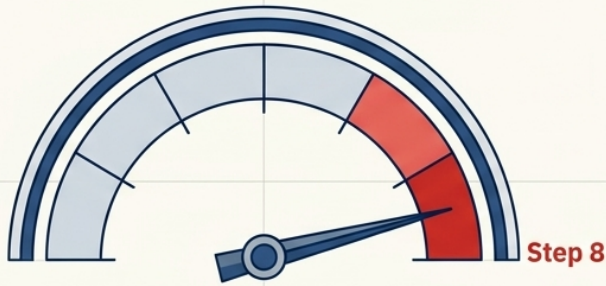


System Diagnostics

- ✓ Executes entirely inside a **single thread** (main thread).
- ✓ **No user requests** are served until **Step 10** is complete.

Macro Realities: Identifying the True Bottlenecks

The Startup Bottleneck (Interview Corner)



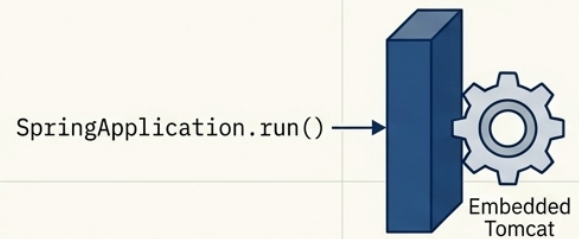
Misconception:

Component Scanning takes the most time.

Reality:

Bean Creation & Dependency Injection (**Step 8**) is the most expensive phase. Scanning reads metadata; Bean Creation actually instantiates objects, resolves dependencies, and applies post-processors.

The Server Startup Illusion (Pitfall)



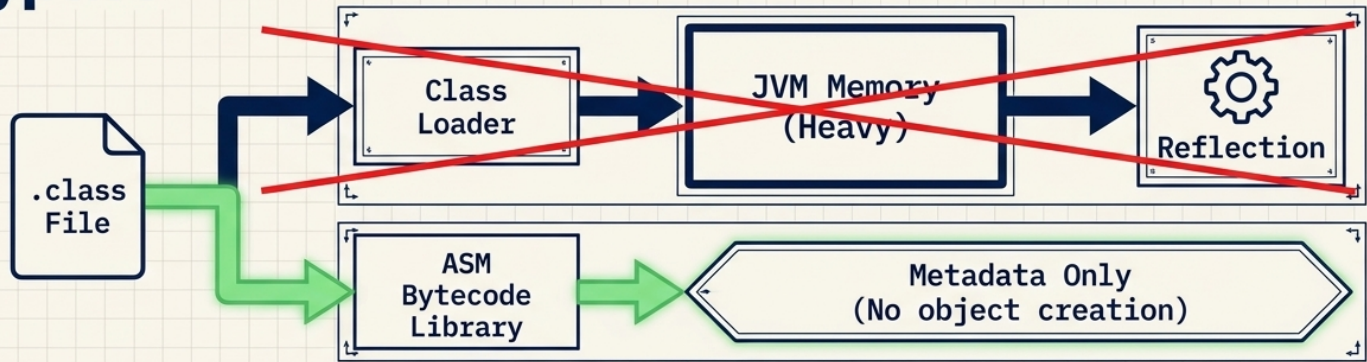
Misconception:

Calling `run()` immediately starts Tomcat.

Reality:

The framework rigorously prepares the Environment, creates the Context, and initializes beans *BEFORE* the embedded server starts accepting requests (Step 9).

High-Speed Scanning: The ASM Bytecode Bypass



During component scanning, Spring does NOT load every class into memory.

By bypassing reflection and reading bytecode directly using ASM, Spring rapidly extracts class metadata. This keeps startup highly efficient, even in monolithic applications with thousands of classes.

The Three Core Actors

The dedicated modules that coordinate the startup sequence.



The Project Manager

SpringApplication

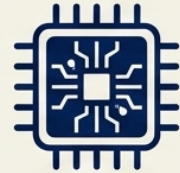
Role: Orchestrator. Plans, coordinates, and ensures the entire sequence fires in the correct order.



The Config Referee

Environment

Role: Arbitrator. Ingests properties from dozens of sources, enforces strict precedence rules, and declares the final configuration values.

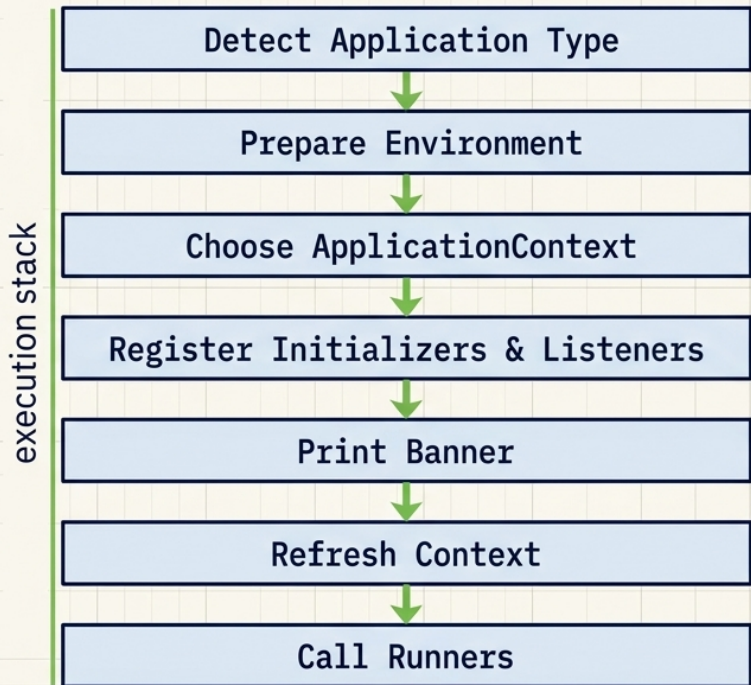


The Operating System

ApplicationContext

Role: The Engine Room. Manages the lifecycle, hosts the beans, and provides enterprise services (events, resources) to the running application.

The Orchestrator: Inside SpringApplication



Source Code Mapping

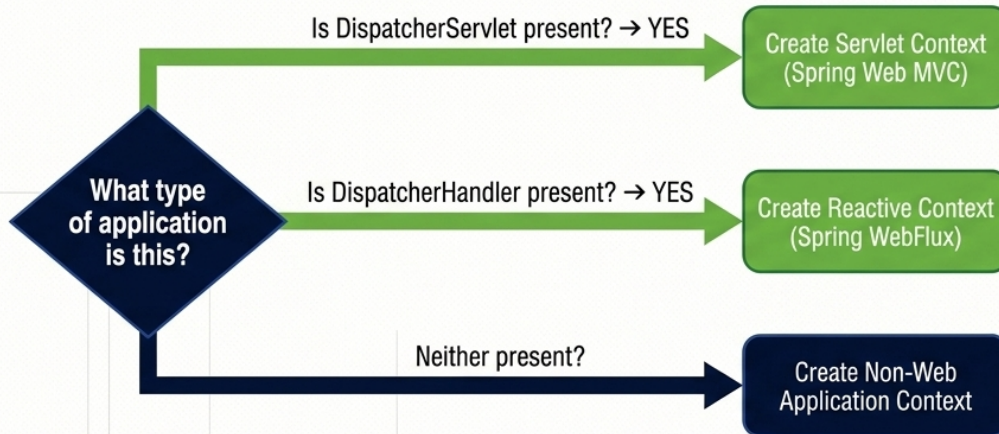
```
prepareEnvironment(...)  
createApplicationContext(...)  
refreshContext(...)  
callRunners(...)
```

Production Tip



Don't customize startup flow by changing `main()`. Use hooks like `ApplicationRunner` or `CommandLineRunner` (which run AFTER context is loaded) to keep `main()` clean.

Dynamic Context Selection: How Spring Knows What to Build

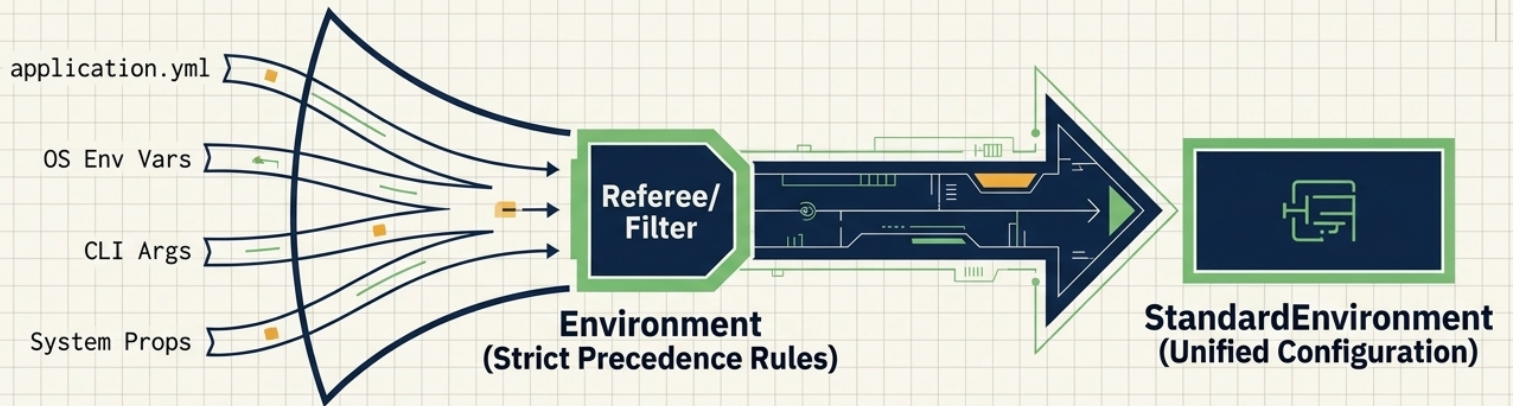


Interview Corner

Who is responsible for selecting the ApplicationContext implementation?

Answer: SpringApplication. The context does not build itself; the orchestrator evaluates the classpath and decides its shape before it is born.

The Config Referee: Unified Environment Resolution



One property. Many places. One winner.

Property Source Precedence Matrix (Highest to Lowest)

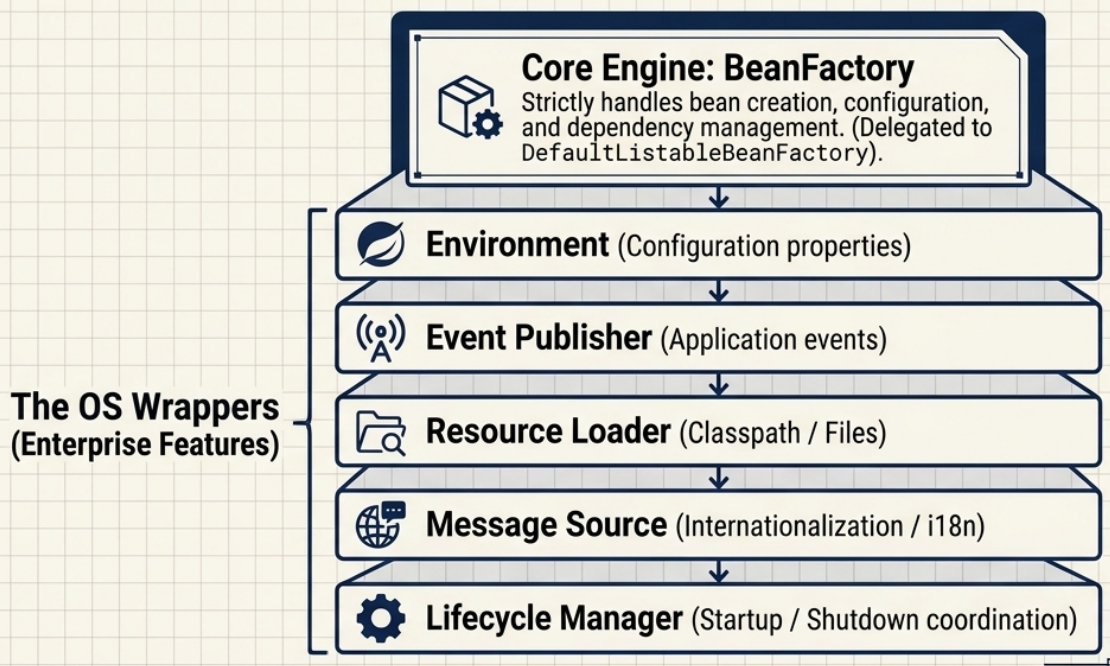
	Command-Line Arguments java -jar app.jar --app.name=prod	Spring Framework *Always Wins*
2	Java System Properties -Dapp.name=staging	Spring Framework
3	Environment Variables APP_NAME=uat	Spring Framework *Crucial for Containers*
4	Profile-Specific Config application-prod.yml	Spring Framework
5	Default Config (application.yml)	Spring Framework
6	RandomValuePropertySource (random.*)	Spring Framework
7	Config Data (Imported via Spring Cloud Config, Vault)	Spring Framework
8	@PropertySource (Manually added annotations)	*Lowest Priority*

Pitfall Alert



Relying only on application.yml and forgetting that leftover OS environment variables will silently override your file configurations.

The Operating System: Inside the ApplicationContext



Production Context

Many production features (graceful shutdown, dynamic config reading, domain events) rely entirely on these outer OS layers, not the internal bean factory.

Container Evolution:

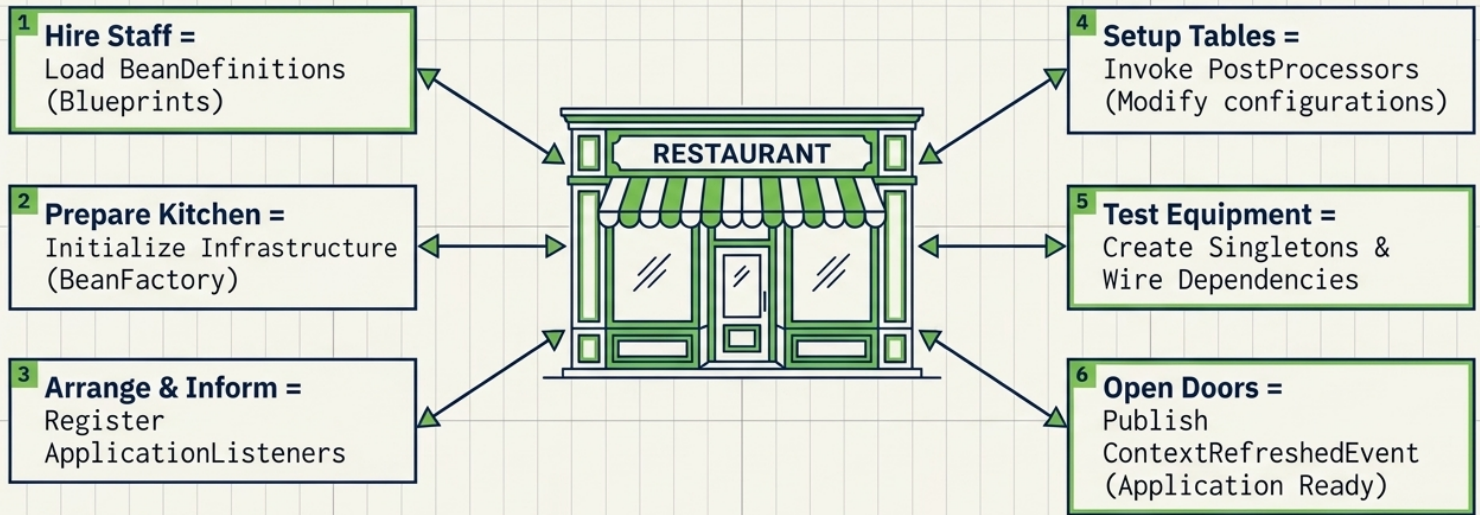
BeanFactory vs. ApplicationContext

	BeanFactory (The Engine)	ApplicationContext (The OS)
Primary Role	Manages beans and DI.	Manages the entire application lifecycle.
Initialization	Naturally lazy (loads beans when requested).	Pre-instantiates singletons eagerly during startup.
Feature Set	Provides basic wiring.	Adds AOP integration, message resource handling, and event publication.
Developer Interaction	Developers rarely interact with it directly.	The standard interface for modern Spring Boot applications.

The “Aha!” Detail

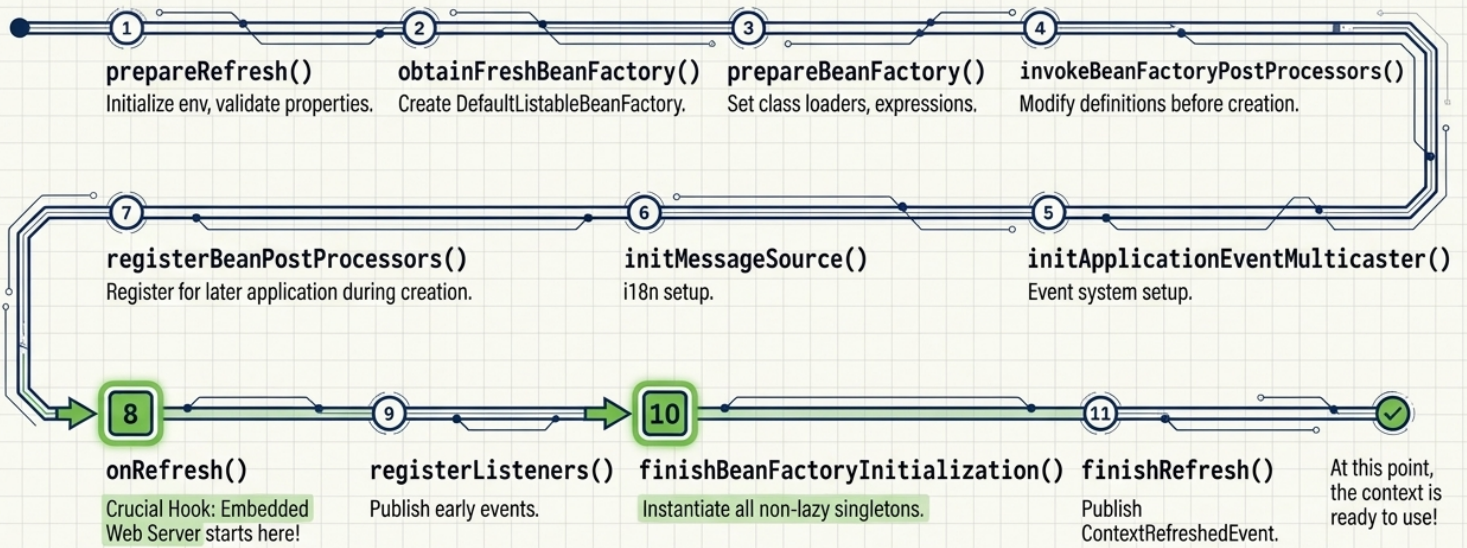
Most developers think ApplicationContext CREATES beans. It doesn’t. It delegates bean creation to `DefaultListableBeanFactory` while wrapping it in higher-level enterprise services.

The Heart of Spring: `AbstractApplicationContext.refresh()`



Contextual Note: You almost never call `refresh()` directly. It is invoked automatically by `SpringApplication`, transforming an empty container into a fully functional system.

The 12-Step refresh() Sequence



org.springframework.context.support.AbstractApplicationContext@refresh ()

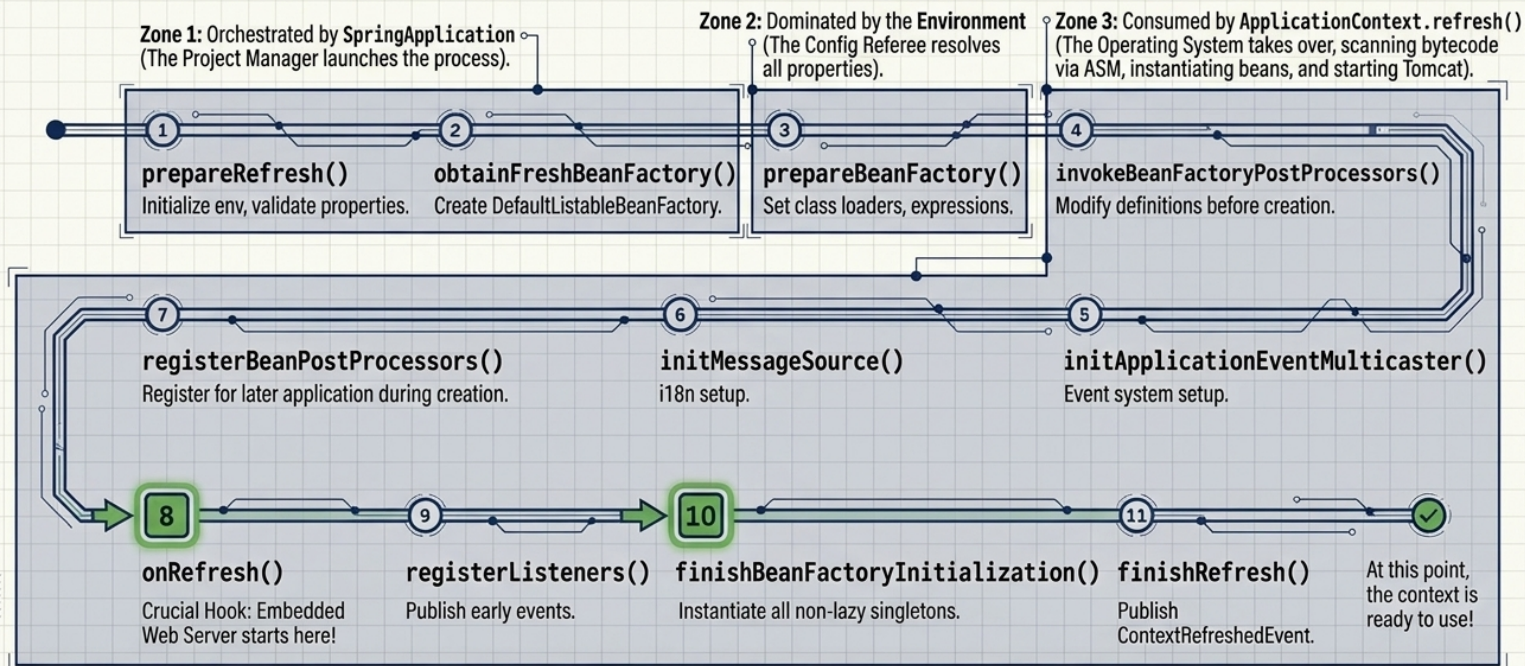
The Idempotency & Safety Contract

The Rules of refresh()

1. **Idempotent:** Can be called only once.
2. **Ordered:** Guarantees strict ordering of infrastructure initialization.
3. **Atomic State:** Ensures the context is in a perfectly consistent state before serving traffic.



System Anatomy: The Unified Startup Overlay



The "magic" of Spring Boot is simply a strictly ordered hand-off between these three specialized components.

`org.springframework.context.support.AbstractApplicationContext@refresh()`

The Developer's Diagnostic Cheat Sheet

Production Best Practices

- Remove unused starters (each introduces dozens of auto-configurations).
- Prefer constructor injection and enable lazy initialization where appropriate to reduce Step 8 time.
- Measure startup bottlenecks using Spring Boot Startup metrics (/actuator/startup).
- Use @ConfigurationProperties for type-safe configuration binding.

Startup Anti-Patterns

Trying to access beans before refresh() completes.

Result: BeanCurrentlyInCreationException or NullPointerException. Let Spring finish its contract.

Placing heavy startup logic in constructors or @PostConstruct.

Fix: Use ApplicationRunner or CommandLineRunner.

Understand the internals. Build better systems.